41052 - Advanced Algorithms

Exact and Approximate Algorithms for Minimum Vertex Cover

Programming Assignment 2 | Theme 1 | Spring 2026

Derek Song - 24833978

James Huang - 24970865

1 The problem and the algorithms **3**
1.1 Problem definition and scope 3
1.2 Research questions 3
1.3 Exact branch and bound 3
1.3.1 Search state and initial solution 3
1.3.2 Branching rule and search order 3
1.3.3 Pruning with upper and lower bounds 3
1.3.4 Correctness and interruption 4
1.4 Maximal matching approximation 4
1.4.1 Key idea and pseudocode 4
1.4.2 Feasibility and approximation guarantee 5
1.4.3 Complexity and implementation decisions 5
2 Theoretical comparison and predictions **5**
2.1 Time complexity and implementation costs 5
2.2 Structural sensitivity and fair interpretation 6
2.3 Proposed predictions before measurement 6
2.4 Preserving the prediction record 6
3 Experimental methodology **6**
3.1 Correctness and fairness 6
3.2 Inputs and generation 7
3.3 Machine and reproduction 7
3.4 Timing and aggregation 8
3.5 Certificates and limits 8
4 Results and analysis **8**
4.1 Completion and validity 8
4.2 Runtime growth 9
4.3 Cover quality 10
4.4 Search effort and structure 11
4.5 Labelling and generalisation 12
5 What surprised us **13**
5.1 Earlier timeouts and the cost of proof 13
5.2 Checking the explanation 13
6 What we would do next **14**
7 Individual contributions **14**
7.1 Derek Song 14
7.2 James Huang 14
8 AI use **15**
8.1 Per member disclosure 15
8.2 Specific AI failures and responses 15
8.3 AI in collaborative work 15
References **16**

1 The problem and the algorithms

1.1 Problem definition and scope

Given a simple undirected graph G = (V, E), a vertex cover is a subset C of V such that, for every edge (u, v) in E, at least one of u or v belongs to C. The minimum vertex cover problem asks for a cover of smallest cardinality. We denote the number of vertices by n, the number of edges by m, and the optimum cover size by OPT(G). The optimisation objective is to minimise |C| subject to covering every edge. This is different from finding a merely inclusion-minimal cover, which cannot have a vertex removed but may still be larger than an optimal cover.

Minimum vertex cover is NP-hard on general graphs (Carnegie Mellon University, 2017). This motivates comparing an exact method, which can establish optimality, with an approximation that trades some solution quality for a faster bounded procedure. NP-hardness concerns worst-case instances; it does not imply that every graph is difficult or that size alone determines practical difficulty.

Our scope is unweighted graphs with vertices identified by integers from 0 to n - 1. Disconnected graphs and isolated vertices are permitted. The shared Graph class rejects self-loops and invalid identifiers, treats reversed endpoints as the same undirected edge, removes duplicate edges, and stores edges in canonical sorted order. These decisions make input semantics consistent across implementations. They also mean that shuffling constructor input does not change the traversal order retained by the graph.

1.2 Research questions

We investigate how size, density, structure, and labelling affect the cost of proving optimality and the approximation's solution quality. We also ask whether search statistics explain runtime better than n and m alone. Conclusions concern these implementations and input families.

1.3 Exact branch and bound

1.3.1 Search state and initial solution

The implemented exact solver represents a search state using the remaining uncovered edges, a bit mask of selected vertices, and the selected-set size. Initially, all vertices incident to an edge form a feasible cover. This is the incumbent, meaning the best complete solution currently known. Isolated vertices are excluded. The initial solution is deliberately simple; it is not supplied by the partner's approximation.

An explicit last-in-first-out stack gives depth-first traversal without Python recursion. Each state retains its own remaining-edge tuple, avoiding changes to a shared graph.

1.3.2 Branching rule and search order

An uncovered edge (u, v) forces every completion to include u or v, possibly both. Two children select the respective endpoints and remove their incident edges. The branches cover every feasible completion but need not be disjoint.

The solver chooses an edge of maximum combined residual endpoint degree, breaking ties by smaller identifiers, then explores its higher-degree endpoint first. This aims to obtain a useful incumbent quickly; it does not guarantee an optimal first choice.

1.3.3 Pruning with upper and lower bounds

Two checks avoid exploring branches that cannot improve the incumbent. First, if the selected-set size is already at least the incumbent size, further additions cannot produce a smaller cover. Second, the solver greedily constructs a matching in the remaining graph. A matching consists of edges with no shared endpoints. Covering these disjoint edges requires at least one different vertex for each matching edge. Its cardinality is therefore a lower bound on the number of additional vertices needed.

If s vertices have been selected, the remaining matching has size L, and the incumbent has size U, the branch can be discarded when s + L >= U. Equality is sufficient because the task is to return one optimum cover, not enumerate all covers of that size. The same matching argument underlies the approximation guarantee discussed below (Dinitz, 2019).

The initial graph's matching size is also retained as a global lower bound. If a complete cover reaches that bound, the solver can stop immediately: a feasible upper bound and a proven lower bound coincide. Otherwise, optimality is established by exhausting all branches that could improve the incumbent.

1.3.4 Correctness and interruption

The incumbent remains feasible, branching preserves every potentially better completion, and pruning removes only states unable to improve it. Exhausting those branches proves optimality. Equality with the global lower bound also proves optimality.

The result separately records the cover, its size, optimality status, lower bound, and search statistics. Status is either optimal, timeout, or node_limit. Interrupted runs return a valid incumbent without claiming that its size equals OPT(G). Their reported lower bound is the initial matching bound and may be weak. An empty graph has optimum zero and can be resolved without visiting a search state.

The time limit is cooperative: it is checked between search states. Setup, a state's processing, and output construction can extend beyond the requested limit. A node limit caps popped states, including states subsequently pruned, rather than only counting states that create children.

1.4 Maximal matching approximation

The approximation is implemented in vertex_cover/approximation.py as solve_approximation(graph). It constructs a greedy maximal matching and returns both endpoints of every selected edge. The solver accepts the same immutable Graph as the exact algorithm, so both methods can be evaluated on identical instances without converting between graph representations.

1.4.1 Key idea and pseudocode

A matching is a collection of edges with no shared endpoints. The implementation scans Graph.edges in its canonical sorted order. A Boolean array records which vertices already belong to the matching. An edge is selected only when both entries are false; its endpoints are then marked. This implements the endpoint-selection approach described by Dinitz (2019) without deleting edges from the graph.

```
solve_approximation(graph):
    matched = [false] repeated graph.num_vertices times
    matching = empty list
    for (u, v) in graph.edges, in canonical order:
        if not matched[u] and not matched[v]:
            append (u, v) to matching
            matched[u] = matched[v] = true
    cover = {v : matched[v] is true}
    return immutable cover and matching
```

For a four-vertex path with sorted edges (0,1), (1,2), (2,3), the solver selects (0,1), skips (1,2) because vertex 1 is marked, and selects (2,3). The matching has size two and the returned cover is {0,1,2,3}. A minimum cover is {1,2}, of size two: the two disjoint outer edges require at least two vertices. The

approximation therefore reaches its worst-case factor of two on this example. On a triangle it selects one edge and returns two endpoints, which is optimal. These are manual controls, not measured performance findings.

1.4.2 Feasibility and approximation guarantee

Matching invariant. Before an edge is selected, neither endpoint is marked. Marking both endpoints prevents any later selected edge from sharing either one. The selected edges therefore remain a matching throughout the scan.

Feasibility and maximality. A selected edge has both endpoints in the returned cover. A skipped edge already had a marked endpoint when examined, and marks are never removed. Hence every input edge has at least one endpoint in the final cover. Every unselected edge also touches a matching endpoint, so no further edge can be added to the matching. This establishes maximality; it does not establish maximum cardinality.

Approximation bound. Let M be the selected matching and C the returned cover. Any vertex cover needs a distinct vertex for every edge in M because matching edges share no endpoints. Thus |M| <= OPT(G). The construction includes exactly two different vertices for each matching edge, giving |C| = 2|M| <= 2 OPT(G), the standard matching-based guarantee (Dinitz, 2019). For an edgeless graph both sets are empty and the inequality still holds, although the ratio 0/0 is undefined.

1.4.3 Complexity and implementation decisions

Initialising the Boolean array and constructing the cover each take O(n) time. The single edge scan takes O(m) time with direct indexed membership checks. Under the conventional RAM model, the complete call therefore takes O(n + m) time and O(n) auxiliary space, excluding the existing graph. The matching contains at most floor(n/2) edges and the cover at most n vertices, so retaining and freezing the result preserves these bounds.

Canonical traversal makes shuffled, duplicated, or reversed constructor edges equivalent after normalisation. Vertex relabelling can still change the matching: the isomorphic path with edges (0,1), (0,2), (1,3) selects only (0,1), returning two vertices rather than four. The final matching is still maximal in both cases. Labelling experiments must therefore save vertex permutations, rather than merely shuffle the input edge list.

2 Theoretical comparison and predictions

2.1 Time complexity and implementation costs

As established in Section 1.4, the approximation takes O(n + m) time and O(n) auxiliary space, excluding graph storage.

For exact search, each branch selects a new vertex and removes its incident edges. Search depth is at most n, and the binary search tree has $O(2^n)$ states in the worst case. At each state, the implementation scans remaining edges for its matching bound, degree counts, branch choice, and child tuples. Under a conventional unit-cost treatment of masks and integer operations, $O((n + m)\ 2^n)$ is a conservative time bound. Pruning reduces visited states on some inputs without changing this exponential upper bound.

Python integers have arbitrary precision (Python Software Foundation, n.d.-a). The selected and matched masks may span multiple machine words, so their operations should not be described as constant-time for arbitrarily large n. If b is the number of machine words needed for an n-bit mask and T states are visited, a conservative model for the implementation's main work is O(T(n + mb) + nb + n + m), assuming usual dictionary costs. This distinction matters when interpreting representation overhead.

The exact depth-first stack has O(n) pending states, but each can hold O(m) edge references and an n-bit selected mask. A conservative storage bound is O(nm + nb + n + m) machine-word units, with additional Python object overhead. An iterative stack avoids a recursion limit; it does not make memory usage constant. The approximation's lower working-storage requirements may therefore matter independently of elapsed time.

2.2 Structural sensitivity and fair interpretation

Exact difficulty depends on incumbent quality, matching bounds, and edges removed by branching. Hubs can remove many edges at once; equivalent choices can make optimality expensive to prove. Size and density alone need not predict difficulty.

Both methods use matchings and label-dependent policies. The exact solver searches for an optimum, while the approximation returns one matching's endpoints. Changing the exact solver's initial cover would define a separate experimental variant and must not occur silently during measurement.

2.3 Proposed predictions before measurement

P1 Runtime advantage. We expect the approximation to be faster on most nontrivial shared instances because it avoids a search tree. On tiny graphs, timing overhead may obscure the difference. We do not predict a systematic crossover at which exact search becomes faster as n increases.

P2 Practical exact search limit. As an initial, low-confidence prediction, some random instances with edge probability 0.3–0.7 and approximately 30–50 vertices may reach a two-second search limit. This is a proposed range to test, not an established threshold. Hardware, bounds, and graph structure may shift it substantially. Finding no interruption in this range would contradict this prediction and should be reported honestly.

P3 Density. Increasing density should increase the approximation's scan work at fixed n. Exact runtime need not increase monotonically because extra edges can strengthen bounds as well as increase processing cost. We expect search-state counts and bound behaviour to help explain departures from a simple density trend.

P4 Known quality extremes. On a nonempty star or a disjoint union of edges, the unmodified approximation should return twice the optimum. On a clique with n vertices, its ratio should be 2 floor(n/2)/(n - 1). These are mathematical controls, not discoveries to present as surprises.

P5 Labelling. Relabelling selected graphs should sometimes change the greedy matching, approximation size, or exact search effort while leaving OPT(G) unchanged. Shuffling the original JSON edge list should have no effect under the current canonical Graph representation.

2.4 Preserving the prediction record

Both members confirm that they reviewed and accepted these AI-assisted predictions before the study. The runner saved the prediction text before warm-up and recorded its SHA256. That snapshot establishes which proposals preceded measurement; it is not independent evidence of student endorsement, which is based on the members' confirmation. We retain the original estimates, including P2, when discussing departures below. No signed acceptance or pre-study acceptance commit is recorded.

3 Experimental methodology

3.1 Correctness and fairness

Both solvers use the same immutable Graph. Each algorithm's tests compare all 1,100 labelled simple graphs on zero through five vertices, plus 300 seeded small random graphs, against an independent subset-enumeration oracle. Approximation checks also validate matching membership, disjoint endpoints, maximality, exact agreement between the cover and matching endpoints, and the factor-two guarantee. Structured cases, invalid arguments, disconnected graphs, command-line compatibility, and a 100,000-vertex sparse construction are covered. These are correctness tests, not performance observations.

The comparison workflow adds tests for generation, known optima, integrity checks, mixed completion statuses, pairing, relabelling, and plots. All 49 tests passed during development with Python 3.12.14; the earlier 33-test integration run was recorded with Python 3.14. These development interpreters differ from the final study's Python 3.11.9. The original solver files were preserved when the workflow was added. The exact solver received no approximation-derived incumbent, and the approximation received no exact solution or post-processing. Both implementations retain their own result construction and instrumentation costs.

3.2 Inputs and generation

The study contains 508 graphs. There are 400 random G(n,p) graphs: n in {8, 12, 16, 20, 24, 28, 32, 40}, p in {0.1, 0.3, 0.5, 0.7, 0.9}, and seeds 0 through 9 for every configuration. A local random.Random(seed) examines each unordered vertex pair and includes its edge independently with probability p. The same n and seed reuse random draws across probabilities, producing coupled, nested edge sets. We therefore analyse probability groups separately rather than treating all configurations as independent replicates. Realised edge counts and densities are saved.

Forty-eight controls use the same eight sizes for paths, cycles, stars, cliques, disjoint edges, and complete bipartite graphs. Bipartitions have floor(n/2) and ceil(n/2) vertices. These families isolate hubs, repeated components, dense connectivity, and known optima. Sixty additional inputs relabel paths, cycles, and stars at n = 12 and 24 using permutation seeds 100 through 109. The manifest stores each actual permutation and base identifier. Relabellings appear in a separate analysis and are excluded from primary group summaries; they are not independent graph structures.

3.3 Machine and reproduction

The final run used 64-bit CPython 3.11.9, Windows build 26200, an Intel64 Family 6 Model 165 Stepping 5 processor, 12 logical CPUs, and approximately 31.9 GiB RAM. The CPU marketing name was not captured. Garbage collection remained enabled with thresholds (700, 10, 10); plots used Matplotlib 3.11.2. The run lasted approximately 18 minutes 12 seconds on 3 November 2026, including warm-ups and data handling.

Power settings, temperature, and competing background workloads were not recorded, which limits precise timing reproducibility. The captured Git base was 7b2d5fc9e055132d533b1edbbb2d11b2207d7663; the workflow contained uncommitted changes. Per-file SHA256 hashes identify the executed source, so the base commit alone is insufficient to reconstruct it. The submitted repository must retain that source and the study artifacts.

From the repository root, install plotting dependencies and run the study with these commands. Use a fresh output directory because existing results are protected against overwrite.

```
python -m pip install ".[plots]"
python -m unittest discover -s tests -v
python -m vertex_cover experiment --config configs/study.json --output results/study_reproduction
--plots
```

The original run is in results/study. To regenerate summaries and plots without timing again:

```
python -m vertex_cover analyse --run results/study/run --output results/study_analysis_rebuilt
python -m vertex_cover plot --analysis results/study_analysis_rebuilt --output
results/study_plots_rebuilt
```

Inputs, configurations, prediction snapshot, manifest, raw.csv, details.jsonl, environment, and source hashes are retained. Graph identities and analysis should reproduce; exact timings will vary. A separate 23-graph smoke run validated the pipeline and is excluded from this report's comparative measurements.

3.4 Timing and aggregation

The common external timer was time.perf_counter_ns, backed by QueryPerformanceCounter with reported resolution 100 ns; median back-to-back timer calls also took 100 ns. It measured complete solver calls, including setup, immutable result construction, common wrapper overhead, and result-list retention. Graph generation and normalisation, validation, file I/O, and plotting occurred outside the timed interval (Python Software Foundation, n.d.-b).

Every graph received one untimed warm-up per algorithm and three measured repetitions with alternating solver order. If an algorithm's completed warm-up took less than 1 ms, each repetition batched 25 fresh solver calls and divided total elapsed time by 25. Otherwise it timed one call. Each exact invocation had a two-second cooperative limit and no node limit. Batches store every individual result; three repetitions are not three independent graph samples. Shared list-retention overhead can matter for the fastest calls, so small microsecond differences should not be overinterpreted.

We first take medians over repetitions within each graph, then group medians and interpolated quartiles over base graphs with the same family, n, and p. Exact timing summaries include only instances where every measured repetition and batch call proved optimality. Their paired approximation summaries use exactly the same graph subset. Approximation timing on all instances is available separately. Completed timing curves therefore need the completion table to reveal selection bias.

3.5 Certificates and limits

The runner validates covers and approximation matchings and rejects inconsistent certificates. A measured exact certificate from any repetition can establish OPT for quality analysis, even when the graph fails the stricter all-repetitions timing criterion. Ratios use only certified positive optima; interrupted incumbents are never denominators. Edgeless graphs have undefined ratios and are reported separately. Analytic control optima provide additional checks but are not substituted into the main measured-ratio plots when exact search timed out.

Timeouts record actual observed time and status, not a claim of completion at two seconds. Statistics distinguish visited states, expansions, bound prunes, incumbent updates, leaves, and maximum pending stack size. The latter is a state count, not a memory measurement. The post-run audit checked recorded hashes and independently rechecked every saved measured cover against its graph. This checks feasibility, not an independent proof that every returned optimum is globally minimal.

4 Results and analysis

4.1 Completion and validity

The completed run produced 3,048 measurement rows: 508 graphs times two algorithms times three repetitions. Fast-call batching expands these into 55,752 individual measured solver calls. All recorded covers passed an independent feasibility check; no measured output was invalid. Of 1,524 exact measurement rows, 1,180 were optimal and 344 timed out. All 1,524 approximation rows completed. There were no node-limit statuses.

Exact search completed every repetition on 393 of 508 instances (77.4%). Of the 115 remaining graphs, 114 had no measured exact certificate and one completed only one repetition. Random graphs account for 111 incompletely solved instances, and cliques for four. All 60 relabelled inputs and all other structured controls completed. Table 1 separates random completion by size and density; every cell has ten graph seeds.

Table 1 Random graphs completing every exact repetition out of ten seeds per cell

Vertices	p = 0.1	p = 0.3	p = 0.5	p = 0.7	p = 0.9
8	10	10	10	10	10
12	10	10	10	10	10
16	10	10	10	10	10
20	10	10	10	10	10
24	10	10	10	10	0
28	10	10	2	0	0
32	10	7	0	0	0
40	10	0	0	0	0

The distinction between certificates and timing exclusions matters. Graph gnp_n24_p0.9_s7 timed out in repetitions 0 and 2, but proved optimum 20 in repetition 1 at approximately 1.985 seconds. It contributes a valid ratio of 24/20 = 1.2, but no completed runtime summary. There are consequently 394 graphs with some measured exact certificate, including one edgeless graph, leaving 393 defined quality ratios. This prevents both throwing away a valid proof and treating a fragile near-limit run as reliably completed.

4.2 Runtime growth

The approximation was faster on every fully completed paired base instance in this run. The 333 such base instances had a median individual speed ratio of approximately 224, but that pooled number depends strongly on the chosen family mix and excludes hard unfinished graphs. It is not a universal speedup. Figure 1 instead shows matched runtime curves for two probability groups, with interruptions explicitly marked. Shading represents interquartile ranges across graph instances after within-instance aggregation.

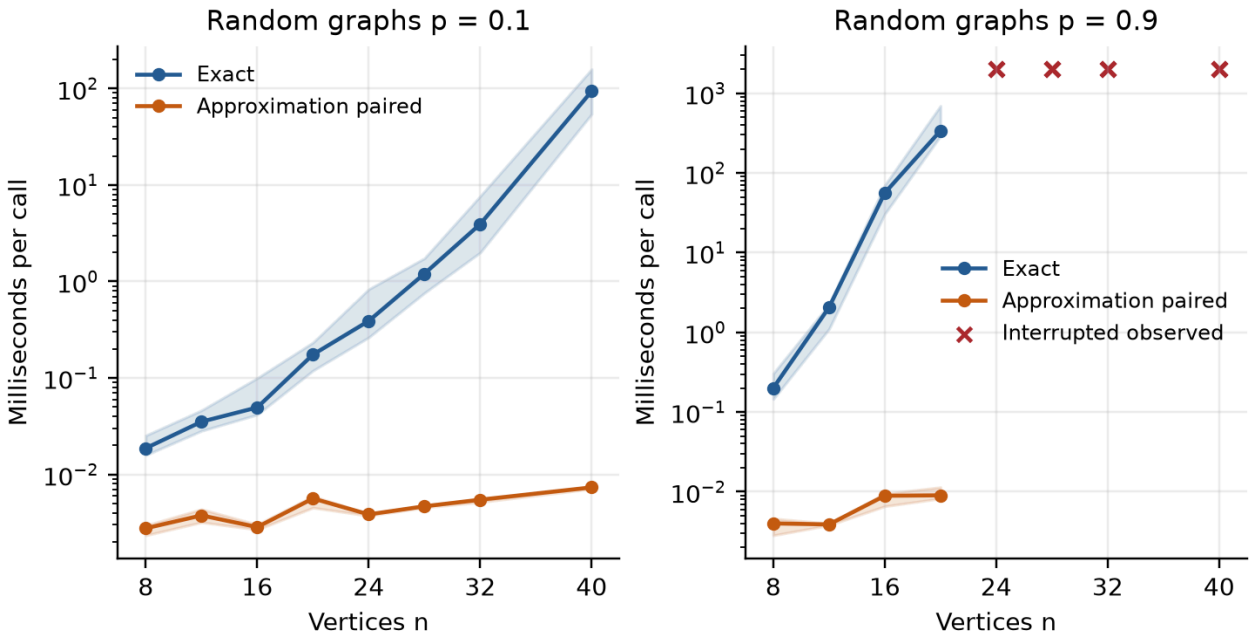


Figure 1 Paired base random runtimes for p = 0.1 and 0.9. Bands show across-instance quartiles. Red crosses show observed interrupted times, not completion times; Table 1 gives denominators.

At p = 0.1, all ten random graphs completed at every size. Exact group medians rose from 0.0187 ms at n = 8 to 93.7 ms at n = 40. At n = 20 and p = 0.9, exact search took a median 338 ms versus approximately 0.00901 ms for the approximation on the same completed set. At n = 24 and p = 0.9, none of the ten graphs completed every repetition. At n = 28 and p = 0.5, only two of ten did; their completed median of 1.264 seconds describes those two survivors, not the whole configuration.

These observations support P1. They qualify P2: timeouts did occur in the proposed 30–50 regime, but also at smaller sizes, including n = 28 at p = 0.5 and n = 24 at p = 0.9. We cannot place a precise boundary between tested sizes, infer a universal threshold, or extrapolate a crossover beyond the sample. Approximation group medians at n = 40 rose from 7.34 microseconds at p = 0.1 to 22.4 microseconds at p = 0.9, consistent with scanning more edges. At small sizes, interpreter and system noise obscure strict monotonic growth.

4.3 Cover quality

All 393 defined measured ratios lie between 1 and 2. Their overall median is 1.5, including relabellings. The 289 certified, nonempty base random graphs have median ratio 1.333. Neither statistic describes all 400 random graphs: certification disproportionately excludes larger, denser hard cases. Figure 2 displays random quality by probability and structured controls separately, retaining the sample counts.

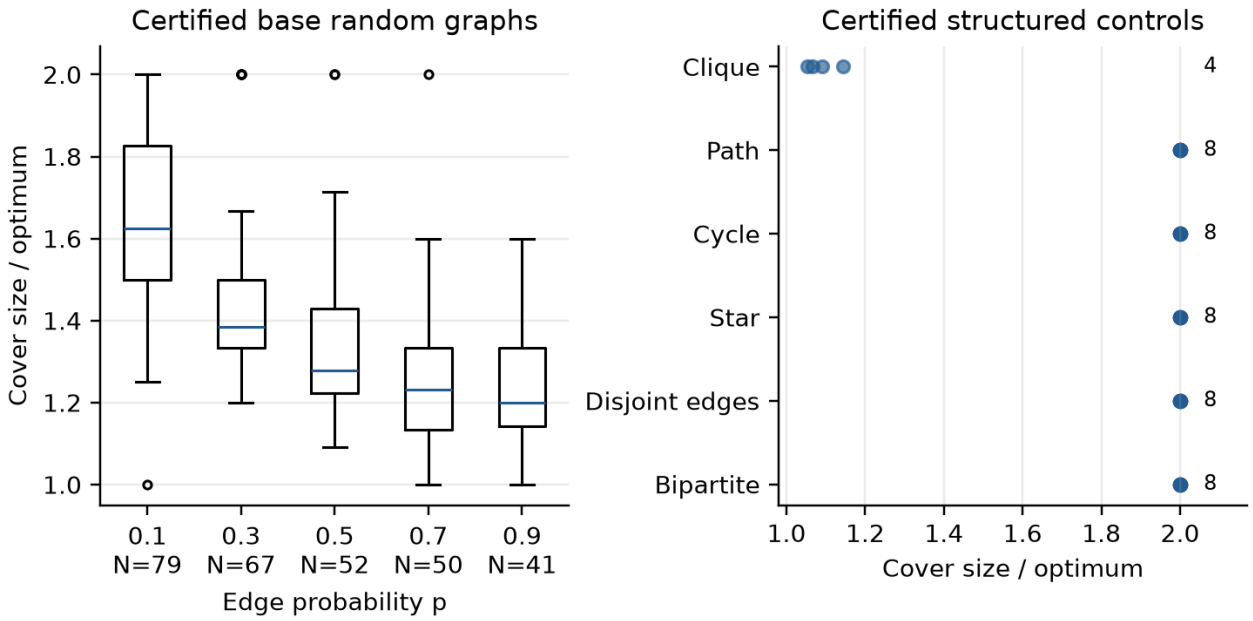


Figure 2 Measured certified quality ratios. Random boxes mix tested sizes descriptively and show sample counts; structured points exclude relabellings and uncertified controls. Right-hand numbers are sample counts.

All base stars, paths, even cycles, disjoint-edge graphs, and balanced complete bipartite controls returned twice the optimum. For the tested canonical paths and cycles, the greedy scan found a perfect matching and therefore selected every vertex, although half suffice. On a star it selected the centre and one unnecessary leaf. These cases validate the algorithm's known bound, as anticipated by P4; they are not unexpected discoveries.

The four cliques with measured exact proofs had ratios from 8/7 to 20/19. Their approximation returned all n vertices, only one more than the optimum n - 1. Larger clique optima are known analytically, but their ratios remain absent from the measured-certificate chart after exact timeouts. This conservative plotting rule should not be mistaken for uncertainty about the mathematical clique optimum. It keeps measured certificates and analytical controls distinguishable.

4.4 Search effort and structure

Table 2 compares concrete controls. The 20-vertex clique visited 524,289 states and took approximately 1.471 seconds. The larger complete bipartite graph with 40 vertices and 400 edges visited only 21 states and took 2.247 ms. Its runtime was about 655 times smaller despite having more vertices and edges. A graph's size and density alone therefore miss the behaviour of this branch-and-bound implementation.

Table 2 Selected controls with median measured timings and search counts

Graph	Edges	Exact ms	Approx µs	Visited states
Clique n = 20	190	1,471.055	8.62	524,289
Clique n = 24	276	2,000.035 *	11.18	702,986
Bipartite n = 40	400	2.247	16.38	21
Path n = 40	39	0.263	5.61	21
Star n = 40	39	0.032	2.92	2

*** Interrupted observed time, not time to optimality; other rows completed all repetitions.**

In the balanced complete bipartite case, the root matching has 20 edges and hence lower bound 20. Canonical high-degree choices select one bipartition on the first search path. The first complete cover has

size 20, so the global lower and upper bounds coincide and all pending branches can be discarded. In the 20-clique, the root matching bound is only 10 while the optimum is 19. Finding a cover of size 19 does not close that gap; many branches must still be ruled out.

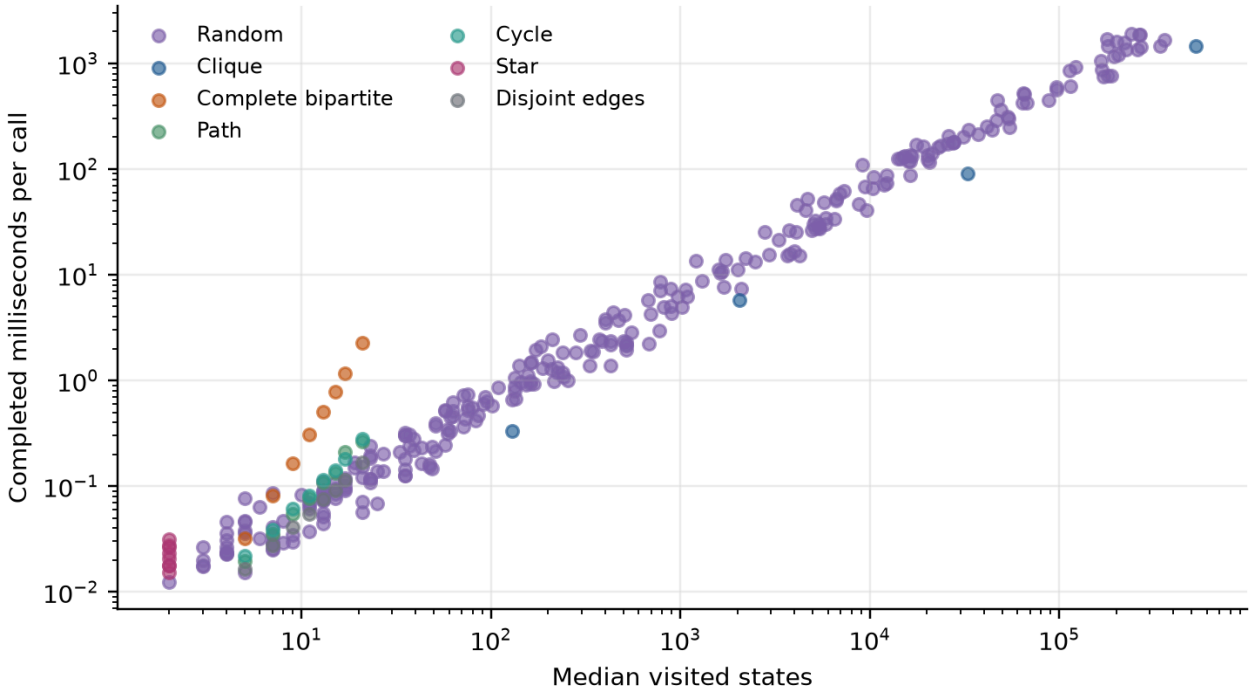


Figure 3 Completed base instances only. Both axes are logarithmic. Visited states explain much runtime variation, while edge work creates different costs per state.

Figure 3 shows that visited-state counts track much of the completed runtime variation, without proving that they alone cause it. At n = 40, the path and balanced complete bipartite graph both visit 21 states, but take 0.263 ms and 2.247 ms respectively. The denser graph requires more edge work per state. The clique's 262,143 matching-bound prunes are substantial in absolute number, yet still leave 262,144 expansions. A large prune count does not by itself mean the bound is effective relative to the total search.

4.5 Labelling and generalisation

Relabelling supports P5. At n = 12, cycle cover sizes across the baseline and ten permutations were 8, 10, or 12, while OPT remained 6. Path covers were 10 or 12 with OPT 6. At n = 24, cycle covers were 20, 22, or 24, and path covers were 20 or 24, with OPT 12 in both families. Stars always returned two vertices. Figure 4 includes all ten permutations rather than just the most favourable one.

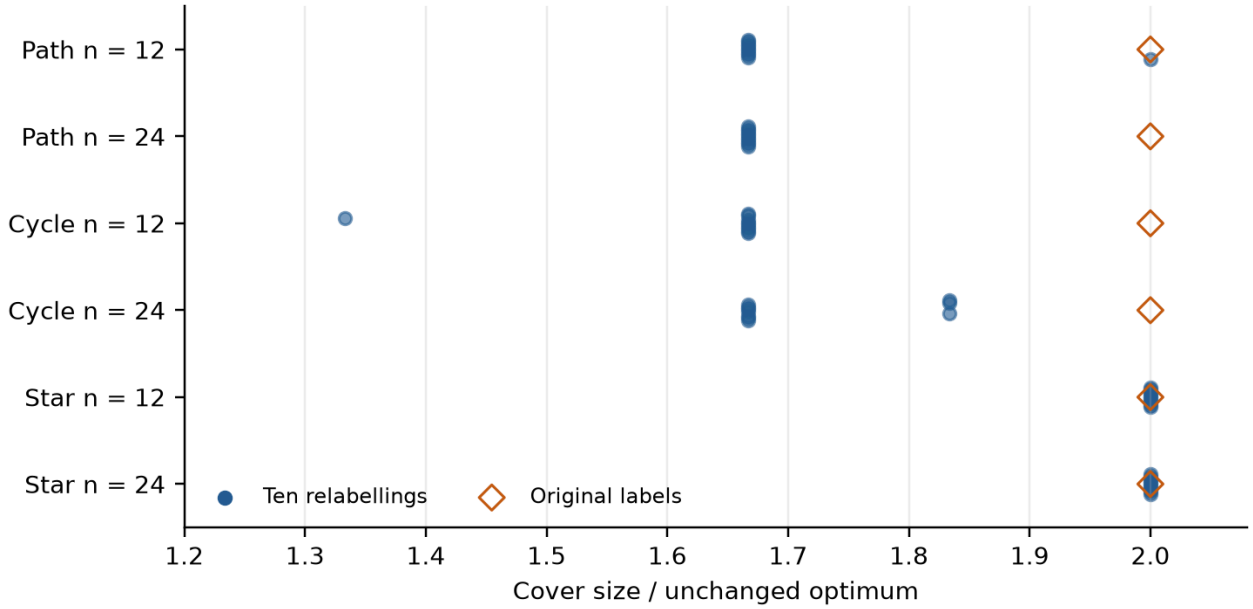


Figure 4 Original labels and all ten permutations per family and size. Small vertical offsets separate overlapping points. The graph optimum is unchanged.

For cycle_n12, exact search visited seven states, versus 25 for cycle_n12_perm100; the approximation cover simultaneously fell from 12 to 10. This is consistent with changes to canonical traversal and exact tie-breaking, not a changed optimisation problem. The same labels are used for both solvers in each comparison. Shuffling an input edge list alone would not create this effect because Graph sorts it.

These results are specific to small synthetic graphs, deterministic policies, one machine, and one implementation pair. Random group sample sizes are ten, whereas each structured size has only one base graph. There is no measured peak-memory comparison. Cooperative limits, unknown background load, and completion selection restrict performance claims. The complete bipartite result does not show that this generic exact solver is an efficient general bipartite solver; it shows that these labelled controls align well with its bound and branch order.

5 What surprised us

5.1 Earlier timeouts and the cost of proof

Relative to the accepted P2 estimate, the practical boundary arrived earlier for dense inputs. The n = 24, p = 0.9 group had no graph that completed all repetitions, while every n = 40, p = 0.1 graph completed. Even within P2's probability range, eight of ten n = 28, p = 0.5 graphs failed the completion criterion. The unexpected element is the location and structure of the boundary, not the existence of exponential worst-case behaviour.

A more revealing example is clique_n24. All three exact repetitions returned a cover of size 23, which is in fact optimal for a clique, yet all three timed out. Their reported lower bound was still 12. The solver had found the right answer but had not established that no answer of size 22 or less existed. In contrast, complete_bipartite_n40 closed the proof after only 21 visited states. This explains why an apparently small optimisation problem can remain expensive after a good solution is already available.

5.2 Checking the explanation

Post-run inspection checked the individual records rather than relying on the plotted timeout marker. Clique_n24 produced one incumbent update and one accepted leaf in every repetition, while visited counts ranged from 693,912 to 712,934 before interruption. The repeated feasible incumbent and weak

lower bound support a proof bottleneck rather than failure to construct a cover. The smaller completed cliques reinforce this: n = 8, 12, 16, and 20 visited 129, 2,049, 32,769, and 524,289 states respectively, with identical state counts across their measured calls. Each four-vertex increase multiplied the dominant work by approximately 16.

The competing explanation that dense graphs are expensive simply because each state scans more edges is insufficient. Complete_bipartite_n40 has 400 edges, more than clique_n24's 276, but completes quickly because its matching lower bound is tight. Per-state costs still matter, as the path comparison shows. The evidence supports an interaction between search-tree size, proof strength, and edge-processing cost rather than a density-only rule.

The mixed outcome for gnp_n24_p0.9_s7 adds a practical limitation: the same deterministic instance can cross a wall-clock deadline as execution speed varies. Its optimal repetition needed 260,369 visited states; its interrupted repetitions stopped earlier, at 253,098 and 256,745. We should not reinterpret this as random algorithm behaviour. No additional timed experiment or modified solver is presented as part of this follow-up; it is an audit of saved repetitions and the actual branching and pruning rules. Stronger bounds would need a separate measured variant to establish their benefit.

6 What we would do next

The first extension should target the observed gap between finding a cover and proving optimality. An approximation-derived incumbent is easy to test as a separate exact-solver variant, but it is not automatically useful: on even cliques the approximation returns every vertex, matching the baseline's initial cover size. The clique evidence instead motivates testing a stronger lower bound or a clique-specific reduction. Any added computation must be timed, since a tighter bound can cost more than it saves.

A second extension would compare several saved vertex relabellings or alternative matching policies as explicit approximation variants. The cycle example demonstrates a quality difference without changing OPT, making this a focused experiment. Larger application-derived graphs and a separately instrumented peak-memory study would test generalisation. The original implementations, prediction record, and baseline results should remain intact so these extensions can be compared fairly.

7 Individual contributions

7.1 Derek Song

Derek was responsible for the exact solver and coordinated the comparison. He refined the scope, directed the AI-assisted implementation and tests, installed plotting dependencies, and ran the 508-graph study. He reviewed and edited generated outputs, coordinated report integration and repository updates, and prepared the exact-algorithm and results sections for the video with AI assistance. The use of generated code, analysis, diagrams, and prose is disclosed in Section 8.

7.2 James Huang

James owned the 2-approximation algorithm, shared-interface and command-line integration, and approximation tests. He contributed through managed AI assisted implementation and test design of the 2-approximation algorithm, and reviewed AI outputs. He also was responsible for the slide design and scripting of the presentation algorithm explanation, worked examples and algorithm invariants, editing, stitching together and uploading of the presentation video, and reviewed the predictions and results with Derek.

8.1 Per member disclosure

Derek: I used ChatGPT and Codex for planning, algorithm explanations, substantial code and test generation, the experiment workflow, plots, report drafting, and results analysis. AI also assisted with drafting the presentation script and slides, editable graph diagrams, and suggestions for my personal reflection. I directed the work, ran the final study, and reviewed and edited outputs. Codex ran development checks, audited saved measurements, and corrected tool-detected issues. I do not claim independent authorship of generated code or personal discovery of those bugs.

James: I used ChatGPT to assist with my understanding of minimum vertex cover, generating the initial scaffold for the approximation algorithm as well as designing tests for the algorithm. It also helped with checking my report sections and personal presentation script and slide design for technical accuracy and errors. I oversaw the project's progress, rewrote my presentation script, reflected upon AI feedback of my scripts and report sections as well as created the slides for worked examples of my approximation algorithm myself.

8.2 Specific AI failures and responses

First, the initial AI-generated cover validator converted inputs into a set before validating every identifier. Python treats True as equal to 1, so conversion could conceal an invalid Boolean identifier alongside an integer (Python Software Foundation, n.d.-a). Codex corrected validation order and added regression cases for both input orderings. This was a tool-detected code defect, not a claimed student discovery.

Second, an AI-generated relabelling plot used automatic horizontal limits. In the smoke run, only one category was present, so a small display jitter expanded across the whole chart and made the category layout misleading. Visual inspection identified the problem. Codex set explicit categorical limits and regenerated the plot from unchanged raw data. Numerical validation alone would not have caught that presentation error.

Third, during the post-run audit, Codex initially compared the saved manifest's raw file hash against a hash recorded for canonical JSON. The mismatch suggested an integrity problem because indentation and whitespace change raw bytes. Inspection of the hashing code showed that the correct comparison sorts JSON keys and uses compact separators before hashing. Repeating that comparison matched the recorded hash; no data file was altered. This was an unhelpful verification step, not evidence of corrupted measurements or a solver failure.

8.3 AI in collaborative work

AI supplied a common interface and explanatory draft, which helped connect separately developed algorithm work. The chat record shows a practical coordination problem as well: the report still described the generator and benchmark command as unimplemented after the workflow existed. Comparing the draft with source files and saved metadata was necessary to bring the report up to date. AI-generated prose therefore required the same revision discipline as generated code.

Both members reviewed and accepted the predictions, while the runner preserved the earlier text. AI helped analyse the outcomes, but that does not turn post-run explanations into prior beliefs. The validator and chart corrections show why checks should include independent small-instance oracles, certificate inspection, and visual review. The manifest example shows that a verification script can itself be wrong.

References

Carnegie Mellon University. (2017, March 28). *Lecture 18: Approximation algorithms* [Lecture notes]. https://www.cs.cmu.edu/~15451-s17/lectures/lec18-approx.pdf

Dinitz, M. (2019, January 29). *Intro, vertex cover* [Lecture notes]. Johns Hopkins University. https://www.cs.jhu.edu/~mdinitz/classes/ApproxAlgorithms/Spring2019/Lectures/lecture1.pdf

Python Software Foundation. (n.d.-a). *Built-in types*. Python 3.11 documentation. https://docs.python.org/3.11/library/stdtypes.html

Python Software Foundation. (n.d.-b). *time — Time access and conversions*. Python 3.11 documentation. https://docs.python.org/3.11/library/time.html